



ONE SEMESTER PROJECT



FINAL DOCUMENT

24/06590

KALOKI CARRICK MWELII

BDS 3106

BACHELOR OF SCIENCE IN DATA SCIENCE

SUPERVISOR - LISPER KENDI

PROJECT TITLE : SMART BANKLYTICS

An Intelligent Data – Driven Bank Management and Customer Analytics System

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor, **Lisper Kendi**, for her continued guidance, support, and valuable feedback throughout the development of this project. Her insights and direction were instrumental in shaping Smart Banklytics into a complete and functional system.

I also extend my appreciation to the faculty and staff of the **Department of Data Science & AI at KCA University** for providing the academic foundation and resources necessary to undertake this project.

Finally, I am grateful to my family and friends for their unwavering encouragement and support throughout this journey.

Table of Contents

ACKNOWLEDGEMENTS	2
GLOSSARY	6
CHAPTER ONE	8
1.0 Background	8
1.1.1 Introduction	8
1.1 Project Overview	8
1.2 Purpose of This Document.....	8
2.0 Problem Statement	8
3.0 Objectives.....	9
3.1 Main Objective	9
3.2 Specific Objectives	9
4.0 Significance of the project	9
4.1 Academic Significance	9
4.2 Industry and Social Significance.....	10
4.3 Technical Significance.....	10
CHAPTER TWO	10
2.1 Literature Review	10
2.1.1 Overview of Banking Information Systems.....	10
2.1.2 Fraud Detection in Financial Systems	11
2.1.3 Analytics Dashboards in Banking	11
2.1.4 Web-Based Application Frameworks for Financial Systems	11
2.1.5 PDF Report Generation in Financial Applications	11
2.2 System Requirements Specifications	12
2.2.1 Functional Requirements	12
2.2.2 Non-Functional Requirements	13
CHAPTER THREE	14
3.1 Data Collection Methods	14
3.1.1 Secondary Research	14

3.1.2 System Observation and Benchmarking	14
3.1.3 Synthetic Data Generation	14
3.1.4 Iterative Developer Testing	14
3.2 SYSTEM DESIGN	15
3.2.1 Architecture	15
3.2.2 Database Design.....	15
3.2.3 Technology Stack.....	17
3.3 Test Plan	17
3.3.1 Test Objectives	17
3.3.2 Test Approach	18
3.3.3 Test Environment	18
3.3.4 Test Case Summary Table.....	18
3.4 Implementation Plan.....	19
3.5 System modules and implementation	20
3.5.1 User Authentication Module	20
3.5.2 Customer Dashboard	20
3.5.3 Transaction Processing Module	21
3.5.4 Fraud Detection Module.....	21
3.5.5 Fraud Alert Management Module	21
3.5.6 Administrator Management Module.....	22
3.5.7 Analytics and KPI Dashboard	22
3.5.8 Password Recovery Module.....	22
3.5.9 PDF Statement Generation Module.....	22
CHAPTER FOUR	23
4.1. Graphical User Interface Test Results.....	23
4.2 Database Test Cases	24
4.3 System Output Test Cases	25
CHAPTER FIVE	25
5.1 Conclusions	25

5.2 Contributions	26
5.3 Recommendations	27
REFERENCES	27
APPENDICES	28
APPENDIX A – SYSTEM SCREENSHOTS	28
Figure 1 — Login Page	28
Figure 2 — Customer Dashboard	29
Figure 3 — Withdrawal Page	29
Figure 4 — Transfer Page	30
Figure 5 — PDF Bank Statement	30
Figure 6 — Admin Dashboard	31
Figure 7 — Analytics Charts	31
Figure 8 — View and Manage User Page	32
Figure 9 — Fraud Alert Review	33
Figure 10 — Create Account Page	33
APPENDIX B – DATABASE SCHEMA	34
Users Table	34
Transactions Table	34
Fraud Alerts Table	35
APPENDIX C – SYSTEM ROUTES REFERENCE	35
APPENDIX D — USER TRAINING MANUAL	36
PART A: CUSTOMER GUIDE	36
PART B: ADMINISTRATOR GUIDE	38
APPENDIX V — SAMPLE SYSTEM CODE	40
V.1 Fraud Detection Function (check_fraud)	40
V.2 Atomic Transfer Route (Flask)	41

GLOSSARY

Term	Definition
Flask	A lightweight Python web framework used to build web applications and APIs
SQLite	A lightweight relational database management system that stores data in a single file
Python	A high-level general-purpose programming language used as the primary development language for this project
HTML	HyperText Markup Language — the standard language for creating web pages
CSS	Cascading Style Sheets — used to style and format web page layouts
JavaScript	A scripting language used to add interactivity and dynamic behaviour to web pages
API	Application Programming Interface — a set of rules that allows software components to communicate
KPI	Key Performance Indicator — a measurable value used to evaluate system or business performance
Dashboard	A visual interface displaying real-time data summaries, metrics, and charts
Session	A temporary server-side record that maintains a user's authenticated state across multiple requests
Authentication	The process of verifying a user's identity using credentials such as username and password
Authorization	The process of determining what an authenticated user is permitted to access or perform
Role-Based Access Control	A security mechanism that restricts system access based on the user's assigned role (customer or admin)

Term	Definition
Transaction	A financial operation such as a deposit, withdrawal, or transfer recorded in the system
Fraud Detection	An automated mechanism that monitors transaction patterns and flags suspicious activity
Fraud Alert	A system-generated notification created when a transaction triggers a fraud detection rule
Account Suspension	The temporary deactivation of a user account triggered by fraud detection or admin action
PDF	Portable Document Format — a file format used to present documents in a fixed layout
fpdf	A Python library used to generate PDF documents programmatically
SQLite Row Factory	A SQLite configuration that returns query results as dictionary-like objects for easier data access
Anomaly Detection	The identification of patterns in data that deviate significantly from expected behaviour
Three-Tier Architecture	A software design pattern separating the system into presentation, application, and data layers
Atomic Transaction	A database operation where all steps are completed successfully together or none are applied at all
BytesIO	A Python in-memory binary stream used to generate files without saving them to disk
Chart.js	A JavaScript library used to render interactive charts and graphs in web browsers
Pandas	A Python data analysis library used for data manipulation and processing
NumPy	A Python library used for numerical computations and array operations

CHAPTER ONE

1.0 Background

1.1.1 Introduction

The rapid growth of digital banking has led to increasing volumes of financial transactions being processed daily, generating large datasets that hold valuable insights into customer behaviour, transaction trends, and overall financial performance. Despite these advancements, many small and medium-scale banking systems remain limited to basic transaction processing, offering little capability for real-time data analytics, fraud detection, or intelligent reporting.

This reality highlighted a significant gap between traditional banking systems and modern data-driven financial platforms. Smart Banklytics was conceived to bridge this gap by combining standard banking operations with intelligent data analytics, automated fraud detection, and interactive reporting within a unified web-based system.

1.1 Project Overview

Smart Banklytics is an intelligent data-driven bank management and customer analytics system developed as a One Semester Project at KCA University. The system is built using Python with the Flask web framework, an SQLite relational database, and a web-based frontend using HTML, CSS, and JavaScript. It serves two primary user categories: customers who manage their bank accounts and perform financial transactions, and administrators who oversee system operations, manage users, review fraud alerts, and monitor banking performance through an analytics dashboard.

1.2 Purpose of This Document

This Final Project Document presents a comprehensive account of the Smart Banklytics project from conception to completion. It consolidates the project proposal, system requirements, design decisions, implementation details, testing outcomes, challenges encountered, and conclusions into a single unified document for submission and evaluation.

2.0 Problem Statement

Many small and medium-scale banking systems lack integrated analytics and fraud detection mechanisms. As a result, financial institutions face significant challenges in monitoring customer behaviour, detecting suspicious transactions, and making informed financial decisions in real time. The absence of data-driven capabilities increases the risk of fraud, poor financial planning, and inefficient service delivery.

Smart Banklytics directly addresses these challenges by developing an intelligent banking platform that integrates transaction management, analytical dashboards, and automated fraud detection into a single cohesive system.

3.0 Objectives

3.1 Main Objective

To develop an intelligent banking system that integrates transaction management, data analytics, and fraud detection using data science techniques.

3.2 Specific Objectives

1. To review and analyze transaction data to generate insights and detect suspicious activities.
 2. To design and implement a secure banking system for user account management and financial transactions.
 3. To collect and store transaction data for analytical processing.
 4. To develop an interactive analytics dashboard for transaction trend visualization.
 5. To develop a rule-based fraud detection system for identifying suspicious transactions.
 6. To generate automated transaction reports and downloadable bank statements.
-

4.0 Significance of the project

Smart Banklytics addresses a critical gap in small and medium-scale banking operations by demonstrating how data science techniques can be applied directly to real-world financial systems.

4.1 Academic Significance

This project demonstrates the practical application of core data science and software engineering concepts acquired throughout the Bachelor of Science in Data Science programme at KCA University. It integrates multiple disciplines including database management, web development, data analytics, and rule-based intelligent systems into a single coherent product, serving as evidence that data science graduates can design and deliver end-to-end intelligent systems independently.

4.2 Industry and Social Significance

The banking sector in Kenya and across East Africa continues to experience rapid digital transformation, yet many community banks, SACCOs, and microfinance institutions still rely on basic transaction systems with no analytical or fraud detection capabilities. Smart Banklytics demonstrates a low-cost, open-source approach to bridging this gap. Specifically, the project:

- Equips financial administrators with real-time KPI dashboards that support faster, data-driven decision-making.
- Reduces financial loss risk by automatically detecting and blocking suspicious transactions before they are committed to the database.
- Empowers customers with transparent access to their transaction history, downloadable statements, and fraud alert notifications.
- Demonstrates that intelligent banking features previously available only to large institutions can be built using lightweight open-source tools such as Flask, SQLite, and Chart.js.

4.3 Technical Significance

Smart Banklytics contributes a working reference architecture for web-based financial systems built on the three-tier model. The implementation of an atomic transfer system, in-memory PDF generation, and a pre-transaction fraud interception function offers practical, reusable solutions to engineering challenges commonly encountered in financial application development.

CHAPTER TWO

2.1 Literature Review

2.1.1 Overview of Banking Information Systems

The evolution of banking information systems has progressed from isolated ledger-based record keeping to integrated networked platforms capable of real-time processing and analytics. Early computerised banking systems focused primarily on automating manual processes such as account balance maintenance and transaction recording. Modern banking systems are expected to deliver real-time analytics, predictive insights, and automated risk management. Smart Banklytics is positioned within this evolution as a system that brings modern data-driven capabilities to a lightweight, accessible platform suited for small and medium-scale financial institutions.

2.1.2 Fraud Detection in Financial Systems

Financial fraud remains a significant threat to banking institutions globally. Bolton and Hand (2002) classify fraud detection approaches into two broad categories: supervised methods that rely on labelled historical fraud data, and unsupervised methods that detect anomalies without prior labels. Phua et al. (2010) conducted a comprehensive survey of data mining-based fraud detection techniques and found that rule-based systems, while simpler than machine learning models, remain highly effective in environments where fraud patterns are predictable and well-defined. Their findings support the design decision in Smart Banklytics to implement a rule-based fraud detection engine as a first iteration, with a clear pathway to machine learning integration identified in the future recommendations.

2.1.3 Analytics Dashboards in Banking

Research by Chaudhuri, Dayal, and Narasayya (2011) on data warehousing and business intelligence highlights that real-time dashboards significantly improve managerial decision-making by surfacing key performance indicators directly from transactional data. The analytics dashboard in Smart Banklytics draws on this principle by computing KPIs including total deposits, withdrawals, transfers, and overall bank balance directly from the SQLite transactions table on each page load, ensuring administrators always work with current data.

2.1.4 Web-Based Application Frameworks for Financial Systems

Grinberg (2018) describes Flask as a micro-framework well-suited to rapid development of data-centric web applications due to its simplicity, extensibility, and native support for RESTful routing. Several studies on lightweight banking prototypes, including those built for academic and microfinance contexts, have adopted Flask as the backend framework of choice due to its low overhead and ease of integration with Python data libraries such as Pandas and NumPy. Smart Banklytics follows this established pattern.

2.1.5 PDF Report Generation in Financial Applications

The generation of downloadable financial statements is a standard requirement in banking systems. Fpdf2 Contributors (2024) document the fpdf library as a capable tool for programmatic PDF generation in Python, supporting custom layouts, tabular data, and logo embedding. The use of Python's io.BytesIO for in-memory PDF generation, as implemented in Smart Banklytics, is consistent with best practices for web applications where writing files to disk introduces unnecessary storage overhead and security concerns.

2.2 System Requirements Specifications

2.2.1 Functional Requirements

The following functional requirements were identified through analysis of the problem domain and review of similar banking systems:

Requirement ID	Description
FR-01	The system shall allow users to log in using a username and password
FR-02	The system shall redirect users to role-appropriate dashboards upon login
FR-03	The system shall allow customers to view their account balance and transaction history
FR-04	The system shall allow customers to perform withdrawals from their accounts
FR-05	The system shall allow customers to transfer funds to other registered accounts
FR-06	The system shall allow administrators to deposit funds into customer accounts
FR-07	The system shall automatically detect and block large transactions exceeding KES 100,000
FR-08	The system shall automatically detect and block accounts with three or more transactions within a five-minute window
FR-09	The system shall generate a fraud alert record for every blocked transaction
FR-10	The system shall allow administrators to mark fraud alerts as legitimate, investigating, or frozen
FR-11	The system shall allow customers to submit appeals on fraud alerts
FR-12	The system shall allow administrators to create new customer accounts
FR-13	The system shall allow administrators to suspend and reactivate customer accounts
FR-14	The system shall display KPI metrics on the admin dashboard

Requirement ID	Description
FR-15	The system shall allow customers to download their transaction history as a PDF bank statement
FR-16	The system shall allow users to recover their password via identity verification

2.2.2 Non-Functional Requirements

Requirement ID	Category	Description
NFR-01	Security	All protected routes shall verify session authentication before granting access
NFR-02	Security	Role-based access control shall prevent customers from accessing admin routes
NFR-03	Performance	The system shall respond to all user requests within 3 seconds under normal load
NFR-04	Reliability	Transfer operations shall use atomic database commits to prevent partial balance updates
NFR-05	Usability	The interface shall be accessible via standard web browsers without additional plugins
NFR-06	Maintainability	The system shall follow a modular architecture separating authentication, transactions, fraud detection, and analytics into distinct components
NFR-07	Portability	The system shall run on any platform supporting Python 3.x and Flask

CHAPTER THREE

3.1 Data Collection Methods

The development of Smart Banklytics was informed by data and requirements gathered through the following methods:

3.1.1 Secondary Research

A review of existing academic literature on banking information systems, fraud detection techniques, and financial analytics platforms was conducted. Key sources included Bolton and Hand (2002) on statistical fraud detection, Phua et al. (2010) on data mining-based fraud detection, and McKinney (2022) on Python-based data analysis. This review informed the choice of technology stack, the fraud detection rule design, and the KPI metrics surfaced on the admin analytics dashboard.

3.1.2 System Observation and Benchmarking

Existing web-based banking interfaces were reviewed to identify common functional patterns expected by users, including account dashboards, transaction history views, and downloadable statements. This benchmarking exercise informed the user interface layout decisions and the range of transaction filters offered in the PDF generation module.

3.1.3 Synthetic Data Generation

As Smart Banklytics is a development-environment system with no access to real customer data, a set of synthetic test accounts and transactions were created directly in the SQLite database during development. These records simulated realistic banking scenarios including regular deposits, inter-account transfers, large transactions, and rapid successive transactions. The synthetic dataset was used throughout functional testing to validate all 53 test cases documented in the testing summary.

3.1.4 Iterative Developer Testing

Throughout development, requirements were refined iteratively. Each module was tested immediately after implementation, and observed behaviour informed adjustments to business rules — for example, the five-minute window used in the rapid transaction fraud rule was determined through testing to be the most practical threshold for distinguishing legitimate high-frequency activity from suspicious behaviour.

3.2 SYSTEM DESIGN

3.2.1 Architecture

Smart Banklytics follows a **Three-Tier Architecture** separating the system into three distinct layers:

Presentation Layer (Frontend): Developed using HTML, CSS, and JavaScript. Includes all user-facing pages: login, customer dashboard, admin dashboard, transaction pages, analytics display, and report generation interfaces.

Application Layer (Backend): Implemented using Python with the Flask framework. Handles all business logic including authentication, transaction processing, fraud detection, analytics computation, and PDF generation.

Data Layer (Database): Managed using an SQLite relational database (bank.db). Stores all user records, transaction logs, fraud alerts, and system audit data.

3.2.2 Database Design

The system uses three relational database tables:

Users Table

Field	Description
id	Primary key (auto-increment)
username	Unique login name
password	User password
first_name	User first name
last_name	User last name
id_number	National ID number
account_number	Unique bank account number
account_type	Savings / Current / Student
balance	Current account balance
role	user / admin

Field	Description
status	active / suspended

Transactions Table

Field	Description
id	Primary key (auto-increment)
account_number	Linked user account
type	deposit / withdraw / transfer_out / transfer_in
amount	Transaction amount
date	Timestamp of transaction
from_account	Source account (transfers)
to_account	Destination account (transfers)

Fraud Alerts Table

Field	Description
id	Primary key (auto-increment)
user_id	Foreign key referencing users table
account_number	Linked user account
reason	Fraud detection reason
amount	Transaction amount that triggered the alert
date	Timestamp of alert
status	pending / reviewed
action	legitimate / investigating / frozen / appealed
from_account	Source account
to_account	Destination account

Field	Description
type	Transaction type that triggered the alert

3.2.3 Technology Stack

Component	Technology
Backend	Python 3.x, Flask Framework
Frontend	HTML, CSS, JavaScript
Database	SQLite
PDF Generation	fpdf (FPDF Library)
Data Analysis	Pandas, NumPy
Visualization	Chart.js, Matplotlib
Platform	Web-based

3.3 Test Plan

The following test plan was defined prior to system testing to ensure complete coverage of all functional requirements.

3.3.1 Test Objectives

- Verify that all functional requirements specified in Section 2.2.1 are correctly implemented.
- Confirm that fraud detection rules trigger accurately under the correct conditions.
- Validate that the system enforces role-based access control consistently across all routes.
- Ensure that atomic transaction commits prevent partial balance updates during transfers.
- Confirm that PDF statements are generated correctly for all filter combinations.

3.3.2 Test Approach

Black-box functional testing was adopted as the primary testing methodology. Each test case was executed manually by interacting with the system through the web browser interface. Test cases were designed based on the functional requirements documented in Section 2.2.1, with both positive (expected inputs) and negative (invalid or boundary inputs) test scenarios included.

3.3.3 Test Environment

Component	Details
Operating System	Windows 11
Browser	Google Chrome (latest)
Backend	Python 3.x with Flask
Database	SQLite (bank.db)
Server	Flask development server (localhost:5000)

3.3.4 Test Case Summary Table

Test Case ID	Module	Description	Input	Expected Result
TC-01	Authentication	Valid login — customer	Correct username and password	Redirected to customer dashboard
TC-02	Authentication	Valid login — admin	Admin credentials	Redirected to admin dashboard
TC-03	Authentication	Invalid password	Wrong password	Error message displayed, no login
TC-04	Authentication	Access protected route without login	Direct URL navigation	Redirected to login page
TC-05	Withdrawal	Valid withdrawal	Amount within balance	Balance deducted, transaction logged

Test Case ID	Module	Description	Input	Expected Result
TC-06	Withdrawal	Insufficient balance	Amount exceeds balance	Error message, no deduction
TC-07	Withdrawal	Large transaction fraud trigger	Amount > KES 100,000	Account suspended, fraud alert created
TC-08	Transfer	Valid transfer	Valid receiver account, sufficient balance	Both balances updated atomically
TC-09	Transfer	Transfer to self	Same account number	Error message, transfer blocked
TC-10	Fraud Detection	Rapid transaction rule	3+ transactions within 5 minutes	Account suspended, fraud alert created
TC-11	Fraud Review	Mark alert as legitimate	Admin selects Legitimate	Transaction processed, account reactivated
TC-12	PDF Statement	Download with date filter	Past 30 days filter	PDF generated with correct date range
TC-13	Password Recovery	Correct three-field match	Username, account number, ID match	Password update permitted
TC-14	Password Recovery	Incorrect field	Wrong ID number	Update denied
TC-15	Analytics	Dashboard KPI load	Admin visits dashboard	All KPIs computed and displayed

3.4 Implementation Plan

The Smart Banklytics system was implemented in five sequential phases over the course of one semester.

Phase	Activities	Outcome
Phase 1 — Planning and Design	Requirements gathering, literature review, system architecture design, database schema design	Finalised system design and database schema
Phase 2 — Core Infrastructure	Flask project setup, SQLite database initialisation, user authentication module, session management, role-based routing	Working login system with customer and admin routing
Phase 3 — Transaction Modules	Deposit, withdrawal, and transfer modules with input validation and fraud detection integration	Fully functional transaction processing with pre-commit fraud checks
Phase 4 — Analytics and Reporting	Admin analytics dashboard with Chart.js visualisations, KPI computation, PDF statement generation with BytesIO	Live analytics dashboard and downloadable PDF statements
Phase 5 — Testing and Documentation	Execution of all 53 test cases, bug fixing, screenshot capture, final document preparation	All test cases passed; system ready for submission

3.5 System modules and implementation

3.5.1 User Authentication Module

The authentication module manages secure login and session control. On form submission, user credentials are validated against the users table. Flask sessions maintain the authenticated state across requests. Users are redirected to role-appropriate dashboards: customers to the customer dashboard and administrators to the admin dashboard. All protected routes include session checks that redirect unauthenticated users to the login page.

3.5.2 Customer Dashboard

The customer dashboard retrieves and displays the logged-in user's account details, current balance, complete transaction history ordered by most recent, and any fraud alerts linked to their account. All data is fetched in real time from the SQLite database on each page load.

3.5.3 Transaction Processing Module

Deposit: Administrators deposit funds directly into customer accounts via the admin panel. The system validates the account number and amount, checks for fraud, updates the user's balance, and logs the transaction in the transactions table.

Withdrawal: Customers initiate withdrawals from their dashboard. The system validates the amount, checks for sufficient balance, verifies the account is active, runs the fraud check, deducts the balance, and logs the transaction.

Transfer: Customers transfer funds to other accounts. The system validates both sender and receiver accounts, checks that they are different, verifies sufficient balance, runs the fraud check, atomically updates both balances in a single database transaction, and logs both a transfer_out record for the sender and a transfer_in record for the receiver.

3.5.4 Fraud Detection Module

The check_fraud() function is called before every transaction is committed. It implements two detection rules:

Rule 1 – Large Transaction Detection: Any single transaction exceeding KES 100,000 triggers an immediate account suspension and creates a fraud alert in the fraud_alerts table with the reason "Large transaction detected."

Rule 2 – Rapid Transaction Detection: If an account records three or more transactions within a five-minute window, the account is automatically suspended and a fraud alert is created with the reason "Multiple rapid transactions."

In both cases, the transaction is blocked, the account status is updated to suspended, and a detailed fraud alert record is inserted capturing the transaction type, amount, source account, and destination account for administrator review.

3.5.5 Fraud Alert Management Module

Administrators review fraud alerts from the **Manage User** page. Three actions are available:

- **Legitimate:** The blocked transaction is reconstructed and processed (balance updated, transaction logged), and the account is reactivated.
- **Investigate:** The alert is flagged for further review and the account remains suspended.
- **Freeze:** The account is confirmed as fraudulent and permanently frozen.

Customers can submit appeals on pending, investigating, or frozen alerts. Appeals update the alert action to appealed and notify the administrator through the notification badge on the admin dashboard.

3.5.6 Administrator Management Module

Administrators can create new customer accounts with auto-generated account numbers starting from 301120060001, incrementing sequentially for each new user. They can view all registered users, access individual user management pages, suspend or activate accounts, and deposit funds directly into user accounts.

3.5.7 Analytics and KPI Dashboard

The admin dashboard computes and displays the following KPIs live from the database on each load:

- Total registered users
- Total number of transactions
- Total deposits, total withdrawals, total transfers
- Total bank balance (sum of all user balances)

Transaction data is aggregated by date to generate time-series data for trend charts rendered using Chart.js. Account type distribution data is retrieved and rendered as a chart showing the breakdown of Savings, Current, and Student accounts.

3.5.8 Password Recovery Module

The password recovery module verifies user identity by simultaneously matching the submitted username, account number, and national ID number against the database. Only on a full three-field match is the password update permitted. The system also validates that the new password and confirm password fields match before committing the change.

3.5.9 PDF Statement Generation Module

Customers can download their transaction history as a formatted PDF bank statement. The /download_pdf route accepts filter parameters including past 30 days, past 14 days, custom date range, and transaction type. The PDF is generated in memory using io.BytesIO with the fpdf library, incorporating the bank logo, customer details, account information, date range header, and a fully formatted transaction table. The file is delivered directly to the browser as a downloadable attachment without being saved to disk.

CHAPTER FOUR

4.1. Graphical User Interface Test Results

Comprehensive functional testing was conducted across all system modules. A total of 53 test cases were executed covering authentication, transactions, fraud detection, admin management, password recovery, PDF generation, and analytics.

Module	Test Cases Result	
Authentication	6	All Passed
Account Management	3	All Passed
Deposit	5	All Passed
Withdrawal	5	All Passed
Transfer	6	All Passed
Fraud Detection	3	All Passed
Fraud Alert Management	4	All Passed
Admin Management	6	All Passed
Password Recovery	3	All Passed
PDF Download	5	All Passed
Analytics Dashboard	7	All Passed
TOTAL	53	53 Passed

All 53 test cases passed successfully, confirming that the system meets all functional requirements specified in the SRS.

4.2 Database Test Cases

The following table documents the key database-level operations tested during system validation, confirming that all read, write, update, and integrity operations performed correctly across the three database tables.

Test Case ID	Database Operation	Table Affected	Expected Result	Outcome
DB-01	Insert new user record	Users	User saved with unique account number	Passed
DB-02	Read user balance on dashboard load	Users	Correct balance retrieved and displayed	Passed
DB-03	Update balance after withdrawal	Users	Balance reduced by correct amount	Passed
DB-04	Atomic update of both balances during transfer	Users	Sender debited and receiver credited in single commit	Passed
DB-05	Insert transaction record after deposit	Transactions	Transaction logged with correct type, amount, and timestamp	Passed
DB-06	Insert transfer_out and transfer_in records	Transactions	Two records created, one per account involved	Passed
DB-07	Insert fraud alert on large transaction	Fraud Alerts	Alert created with correct reason, amount, and account	Passed
DB-08	Update fraud alert status after admin review	Fraud Alerts	Status updated to reviewed, action recorded correctly	Passed
DB-09	Update user status to suspended on fraud trigger	Users	Account status changed to suspended immediately	Passed
DB-10	Password update after identity verification	Users	New password saved only on full three-field match	Passed

4.3 System Output Test Cases

The following test cases validate the system's key outputs including generated PDFs, fraud alerts, transaction records, and dashboard displays.

Test Case ID	Output Type	Description	Expected Output	Outcome
OUT-01	PDF Statement	Download with "All" filter	PDF containing full transaction history	Passed
OUT-02	PDF Statement	Download with "Past 30 Days" filter	PDF containing only transactions from last 30 days	Passed
OUT-03	PDF Statement	Download by transaction type "Withdrawal"	PDF containing only withdrawal records	Passed
OUT-04	Fraud Alert	Large transaction submitted	Fraud alert record created, account suspended	Passed
OUT-05	Fraud Alert	Rapid transactions submitted	Fraud alert record created, account suspended	Passed
OUT-06	Transaction Record	Successful transfer	Two records created: transfer_out and transfer_in	Passed
OUT-07	Dashboard KPIs	Admin loads dashboard	All six KPI values correctly computed and displayed	Passed
OUT-08	Account Status	Admin suspends account	Account status updated to suspended immediately	Passed

CHAPTER FIVE

5.1 Conclusions

Smart Banklytics has been successfully designed, implemented, tested, and documented as a fully functional intelligent banking system. The project demonstrates the practical application of data science and software engineering principles in the financial domain, integrating transaction

management, real-time fraud detection, analytics dashboards, and automated PDF reporting into a unified web-based platform.

All six specific objectives were achieved:

- ✓ transaction data was reviewed and analyzed to generate insights and detect suspicious activities;
- ✓ a secure banking system was designed and implemented supporting full account management and financial transactions;
- ✓ transaction data is collected and stored for analytical processing; an interactive analytics dashboard was developed for KPI monitoring and transaction trend visualization;
- ✓ a rule-based fraud detection system was implemented to identify and respond to suspicious transactions in real time;
- ✓ automated PDF transaction reports and downloadable bank statements were successfully generated with flexible filtering options.

5.2 Contributions

Smart Banklytics makes the following contributions:

Practical Contribution: The project delivers a fully functional, end-to-end intelligent banking web application built entirely with open-source tools. It demonstrates that a data science student can independently design, implement, test, and document a production-quality financial system within a single semester, and that sophisticated features such as real-time fraud detection and analytics dashboards do not require expensive enterprise infrastructure.

Technical Contribution: The system introduces a pre-transaction fraud interception architecture in which the `check_fraud()` function is invoked before any balance update or transaction log entry is committed. This pattern ensures that no fraudulent transaction is permanently recorded in the database, which is a more robust approach than post-transaction fraud flagging used in simpler systems. The atomic transfer implementation using a single database commit for both sender debit and receiver credit additionally contributes a reliable pattern for financial data integrity.

Educational Contribution: Smart Banklytics serves as a reusable reference project for future data science and computer science students at KCA University who are building financial or analytics-driven systems. The modular architecture, comprehensive test documentation, and clear separation of concerns across nine distinct system modules provide a practical template that can be extended and adapted for different domains.

5.3 Recommendations

Based on the current implementation, the following enhancements are recommended for future development:

- **Password Hashing:** Implement bcrypt or similar cryptographic hashing for all stored passwords.
- **Machine Learning Fraud Detection:** Replace or supplement rule-based detection with trained ML models using Scikit-learn for more accurate anomaly detection.
- **Mobile Application:** Develop a companion mobile application for Android and iOS platforms.
- **Production Database:** Migrate from SQLite to PostgreSQL or MySQL for improved concurrency and scalability.
- **Multi-Branch Support:** Extend the system to support multiple bank branches with branch-specific administration.
- **Email Notifications:** Integrate email alerts for fraud detection events, transaction confirmations, and account status changes.
- **Two-Factor Authentication (2FA):** Add an additional layer of login security using OTP or authenticator apps.

REFERENCES

- Flask Development Team. (2024). *Flask Documentation (3.x)*. Pallets Projects.
<https://flask.palletsprojects.com>
- Python Software Foundation. (2024). *Python 3 Documentation*. <https://docs.python.org/3/>
- SQLite Consortium. (2024). *SQLite Documentation*. <https://www.sqlite.org/docs.html>
- Fpdf2 Contributors. (2024). *fpdf2 — PDF Generation Library for Python*.
<https://pyfpdf.github.io/fpdf2/>
- Chart.js Contributors. (2024). *Chart.js Documentation*. <https://www.chartjs.org/docs/latest/>
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical Science*, 17(3), 235–255.

Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). A comprehensive survey of data mining-based fraud detection research. *arXiv preprint arXiv:1009.6119*.

APPENDICES

APPENDIX A – SYSTEM SCREENSHOTS

Figure 1 — Login Page

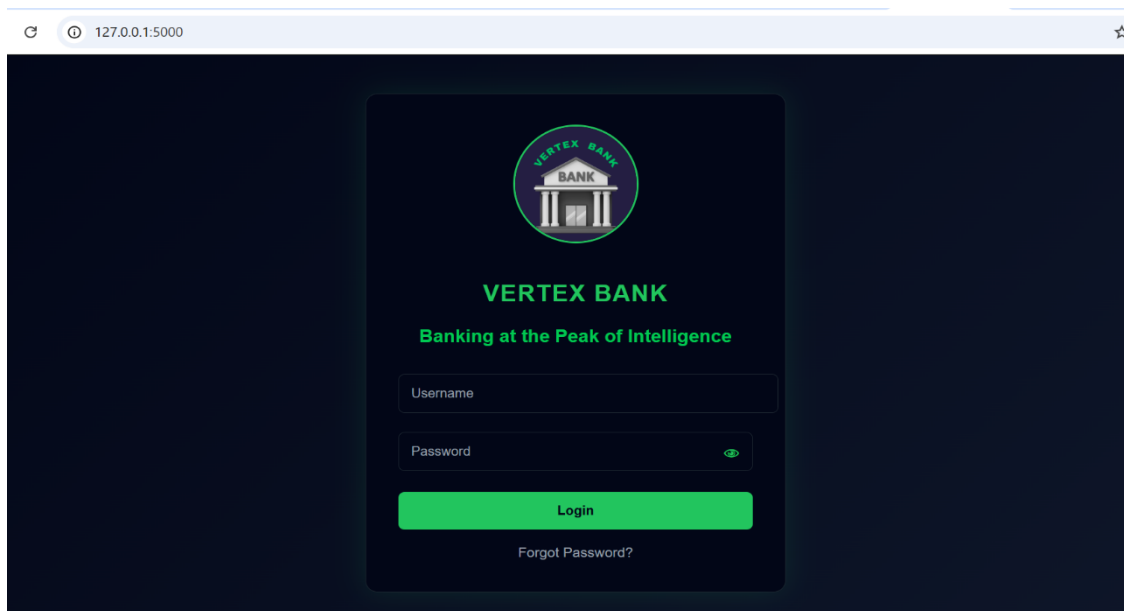


Figure 2 — Customer Dashboard

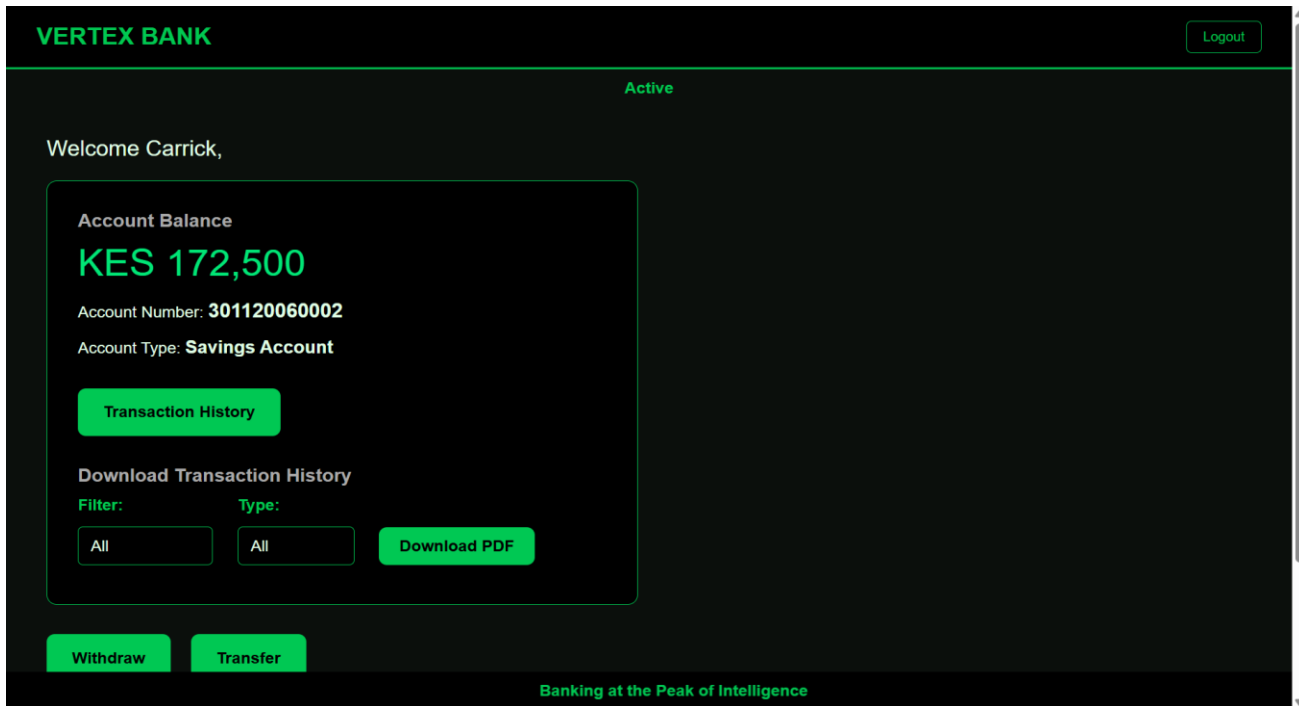


Figure 3 — Withdrawal Page

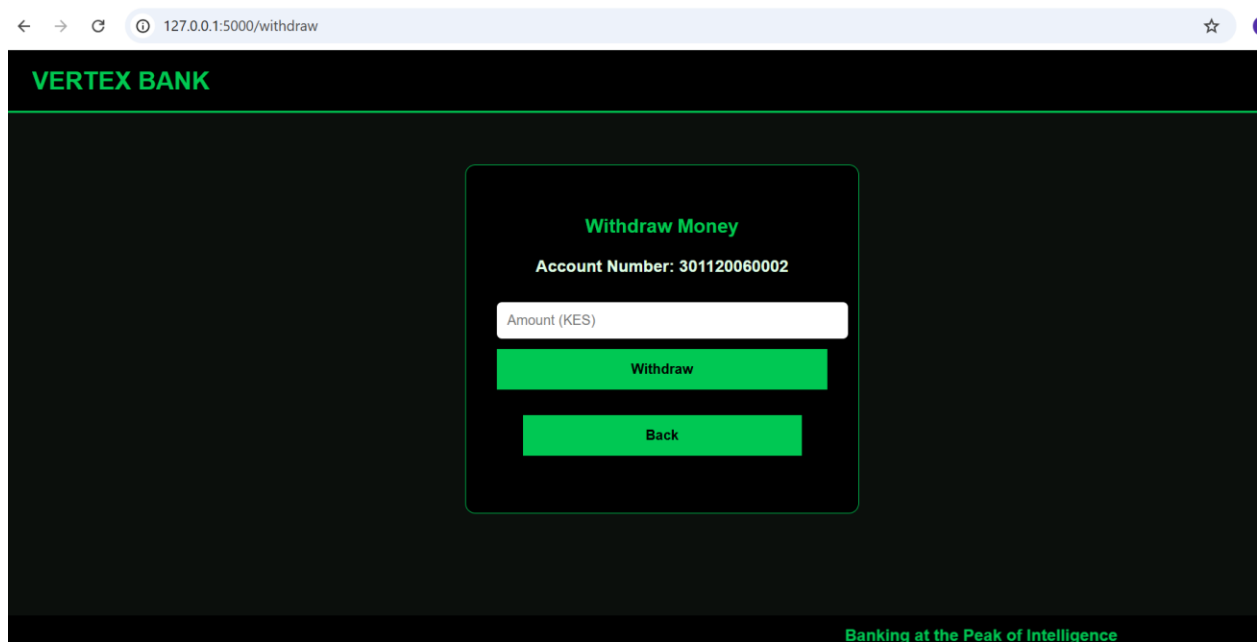


Figure 4 — Transfer Page

127.0.0.1:5000/transfer

☆

VERTEX BANK

Transfer Money

Account Number: 301120060002

Recipient Account Number

Amount (KES)

Send

back

Banking at the Peak of Intelligence

Figure 5 — PDF Bank Statement



Lucy Campos's Transaction History

Account Number: 301120060006 | Balance: KES 53,000

Date	Type	Amount	From	To
2026-03-11 10:20:07	transfer_out	KES 70,000	301120060006	301120060003
2026-03-11 10:19:53	transfer_out	KES 10,000	301120060006	301120060003
2026-03-11 10:19:04	transfer_in	KES 100,000	301120060003	301120060006
2026-02-27 16:35:11	transfer_out	KES 2,000	301120060006	301120060002
2026-02-27 16:01:50	deposit	KES 10,000	-	-
2026-02-10 18:12:00	transfer_in	KES 5,000	301120060005	301120060006

Figure 6 — Admin Dashboard

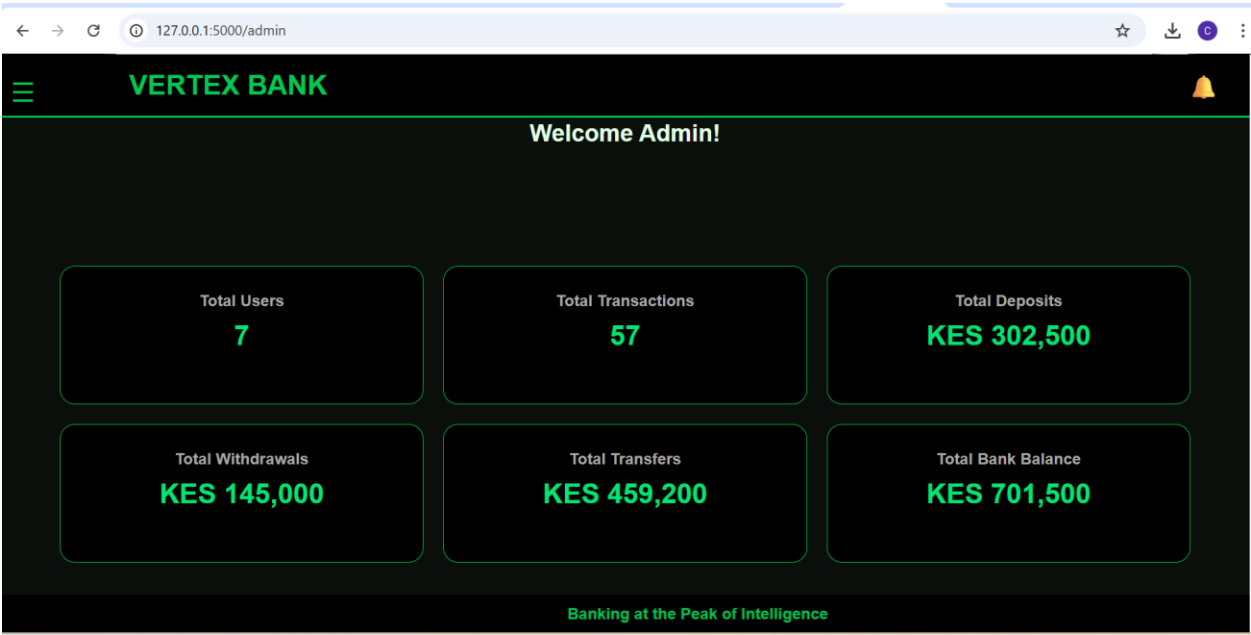


Figure 7 — Analytics Charts

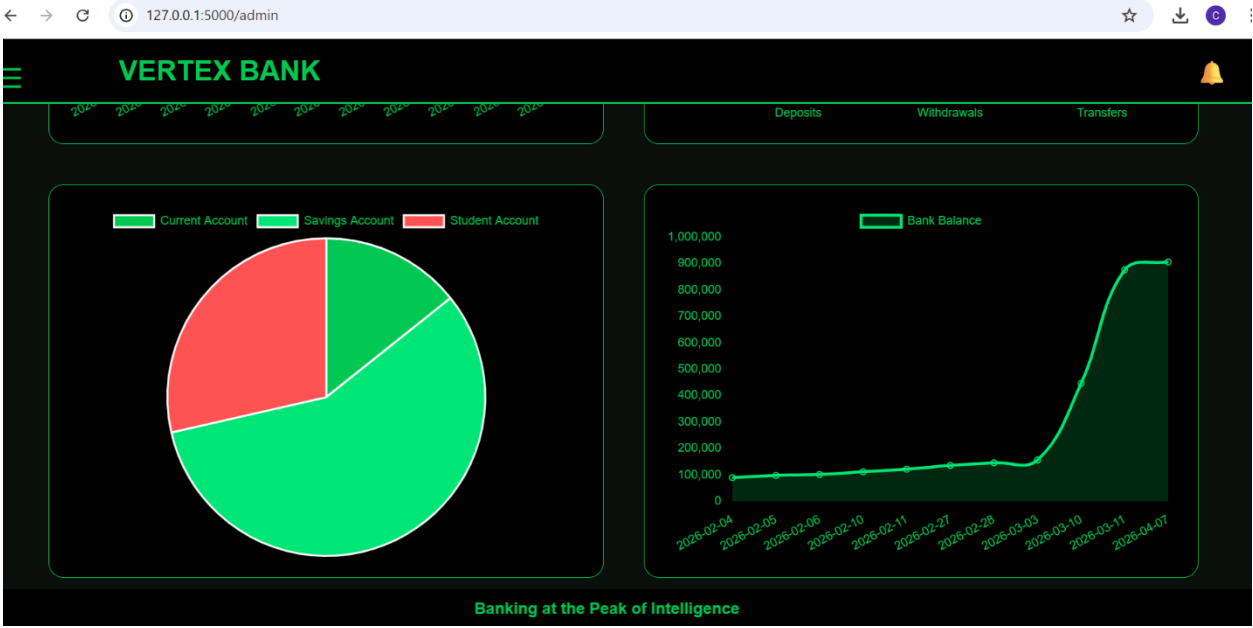




Figure 8 — View and Manage User Page

The 'Admin Users Dashboard' provides a comprehensive view of the system's users. It includes a 'back' button and a 'Create User' button. The table lists seven users with their full names, ID numbers, usernames, account numbers, account types, and current balances in Kenyan Shillings (KES). Each user entry has a 'Manage' button for administrative actions.

Full Name	ID Number	Username	Account Number	Account Type	Balance (KES)	Action
Carrick Mwelii	30112006	admin	301120060001	Savings Account	0	Manage
Carrick Mwelii	30112006	havoc	301120060002	Savings Account	172,500	Manage
John Doe	20260203	johnd	301120060003	Savings Account	86,000	Manage
Mary Jane	20260204	maryj	301120060004	Student Account	30,000	Manage
Franklin Sierra	20260205	frank	301120060005	Current Account	160,000	Manage
Lucy Campos	20260206	lucy	301120060006	Student Account	53,000	Manage
Antony Mchaka	20260227	chaks	301120060007	Savings Account	200,000	Manage

Figure 9 — Fraud Alert Review

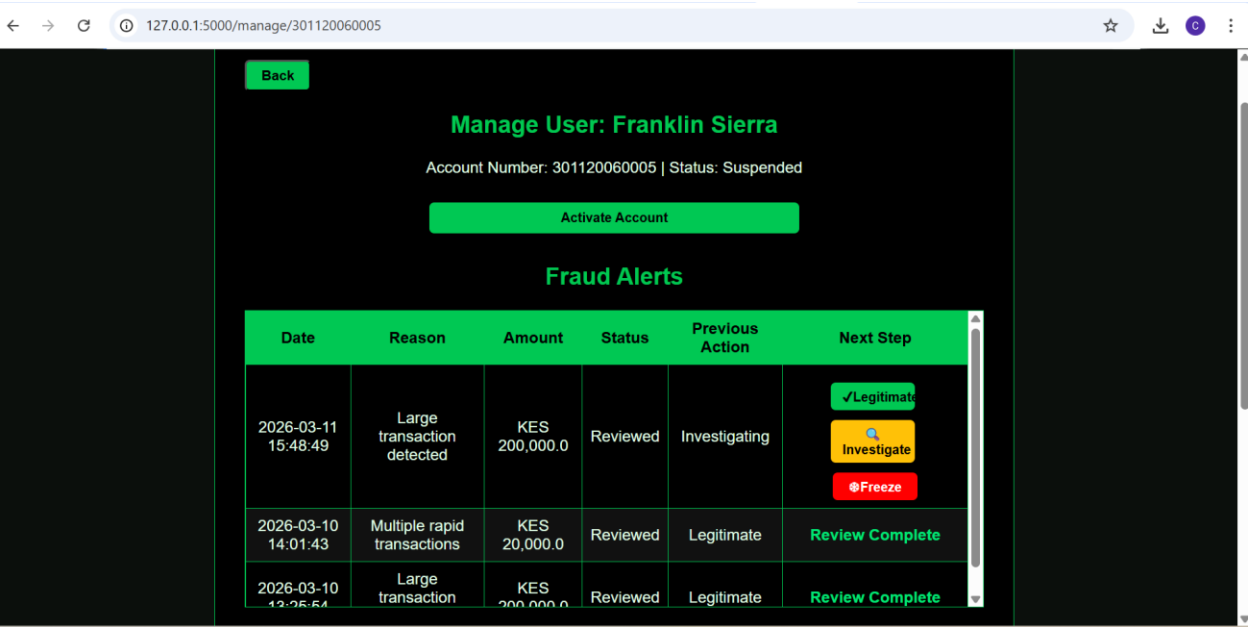


Figure 10 — Create Account Page

The screenshot shows a 'Create Account' form with the following fields and elements:

- First Name
- Last Name
- ID Number(8 digits)
- Username
- Select Account Type (dropdown menu)
- Password
- Confirm Password
- Create Account (button)
- back (link)

APPENDIX B – DATABASE SCHEMA

Users Table

```
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE,  
    password TEXT,  
    first_name TEXT,  
    last_name TEXT,  
    id_number TEXT,  
    account_number TEXT UNIQUE,  
    account_type TEXT,  
    balance INTEGER DEFAULT 0,  
    role TEXT DEFAULT 'user',  
    status TEXT DEFAULT 'active'  
)  
""")
```

Transactions Table

```
CREATE TABLE IF NOT EXISTS transactions (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    account_number TEXT,  
    type TEXT,  
    amount INTEGER,  
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    from_account TEXT,  
    to_account TEXT  
)  
""")
```

Fraud Alerts Table

```
CREATE TABLE IF NOT EXISTS fraud_alerts (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id INTEGER,  
  account_number TEXT,  
  reason TEXT,  
  amount REAL,  
  date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status TEXT DEFAULT 'pending',  
  action TEXT DEFAULT NULL,  
  from_account TEXT DEFAULT NULL,  
  to_account TEXT DEFAULT NULL,  
  type TEXT DEFAULT NULL,  
  FOREIGN KEY(user_id) REFERENCES users(id)  
)  
""")
```

APPENDIX C – SYSTEM ROUTES REFERENCE

Route	Method	Module
/	GET	Login / Home
/login	GET, POST	Authentication
/dashboard	GET	Customer Dashboard
/deposit	GET, POST	Deposit
/withdraw	GET, POST	Withdrawal
/transfer	GET, POST	Transfer
/admin	GET	Admin Dashboard

Route	Method	Module
/signup	GET, POST	Account Creation
/view_users	GET	User Management
/manage/<account_number>	GET, POST	Manage User / Fraud Review
/toggle_status/<account_number>	POST	Suspend / Activate Account
/appeal_alert	POST	Fraud Alert Appeal
/forgot-password	GET, POST	Password Recovery
/download_pdf	GET, POST	PDF Statement Download
/init-db	GET	Database Initialization

APPENDIX D — USER TRAINING MANUAL

Smart Banklytics — User Training Manual *Vertex Bank System — Quick Reference Guide*

PART A: CUSTOMER GUIDE

A.1 Logging In

1. Open your browser and navigate to the system URL (e.g. <http://localhost:5000>).
2. Enter your Username and Password in the fields provided.
3. Click the Login button.
4. You will be taken to your Customer Dashboard automatically.
5. If you have forgotten your password, click Forgot Password? on the login page.

A.2 Viewing Your Account

Your dashboard displays your current Account Balance, Account Number, and Account Type at the top of the page. Below this, your full Transaction History is listed in order from most recent to oldest.

A.3 Withdrawing Money

1. From your dashboard, click the Withdraw button.
2. Enter the amount you wish to withdraw in KES.
3. Click Withdraw to confirm.
4. If your withdrawal is successful, your updated balance will reflect on your dashboard.
5. If the amount exceeds your balance, an error message will appear and no funds will be deducted.

A.4 Transferring Money

1. From your dashboard, click the Transfer button.
2. Enter the Recipient Account Number of the person you wish to send money to.
3. Enter the Amount in KES.
4. Click Send to confirm.
5. Both your balance and the recipient's balance will be updated immediately.

A.5 Downloading a Bank Statement

1. On your dashboard, locate the Download Transaction History section.
2. Select a Filter (All, Past 30 Days, Past 14 Days, or Custom Date Range).
3. Select a Type (All, Deposit, Withdrawal, Transfer).
4. Click Download PDF.
5. A PDF bank statement will download automatically to your device.

A.6 Fraud Alerts

If a transaction you made is flagged as suspicious, your account may be suspended. You will see a Fraud Alert on your dashboard. To dispute this, click the Appeal button next to the alert. Your appeal will be reviewed by an administrator.

A.7 Recovering Your Password

1. On the login page, click Forgot Password?

2. Enter your Username, Account Number, and National ID Number.
3. If all three fields match, you will be allowed to set a new password.
4. Enter your new password and confirm it, then click Update Password.

PART B: ADMINISTRATOR GUIDE

B.1 Logging In as Administrator

Use your administrator username and password on the login page. You will be redirected to the Admin Dashboard automatically.

B.2 Viewing the Admin Dashboard

The Admin Dashboard displays the following live KPIs:

- Total registered users
- Total number of transactions
- Total deposits, total withdrawals, total transfers
- Total bank balance across all accounts

Below the KPIs, transaction trend charts and account type distribution charts are rendered automatically.

B.3 Creating a New Customer Account

1. From the Admin Dashboard, click the menu icon and navigate to View Users.
2. Click Create User in the top right corner.
3. Fill in the customer's First Name, Last Name, ID Number, Username, Account Type, and Password.
4. Click Create Account. The system will automatically assign a unique account number.

B.4 Depositing Funds into a Customer Account

1. Navigate to View Users and click Manage next to the customer.
2. On the Manage User page, enter the deposit amount and confirm.
3. The customer's balance will update immediately.

B.5 Suspending or Activating an Account

1. Navigate to Manage User for the relevant customer.

2. Click Suspend Account to deactivate the account, or Activate Account to reactivate it.

B.6 Reviewing Fraud Alerts

1. Navigate to Manage User for a customer with a pending fraud alert.
2. Review the alert details including the reason, amount, date, and transaction type.
3. Select one of three actions:
 - **Legitimate** — the transaction is processed and the account is reactivated.
 - **Investigate** — the alert is flagged for further review and the account remains suspended.
 - **Freeze** — the account is permanently frozen due to confirmed fraud.

APPENDIX V — SAMPLE SYSTEM CODE

V.1 Fraud Detection Function (check_fraud)

```
def check_fraud(account_number, amount, transaction_type,
                 from_account=None, to_account=None):
    conn = get_db()
    user = conn.execute(
        'SELECT * FROM users WHERE account_number = ?',
        (account_number,)
    ).fetchone()

    # Rule 1: Large transaction detection
    if amount > 100000:
        conn.execute(
            '''INSERT INTO fraud_alerts
               (user_id, account_number, reason, amount, type,
                from_account, to_account)
               VALUES (?, ?, ?, ?, ?, ?, ?)''',
            (user['id'], account_number,
             'Large transaction detected', amount,
             transaction_type, from_account, to_account)
        )
        conn.execute(
            'UPDATE users SET status = ? WHERE account_number = ?',
            ('suspended', account_number)
        )
        conn.commit()
        return True # Fraud detected - block transaction

    # Rule 2: Rapid transaction detection (3+ in 5 minutes)
    five_minutes_ago = datetime.now() - timedelta(minutes=5)
    recent = conn.execute(
        '''SELECT COUNT(*) as cnt FROM transactions
           WHERE account_number = ? AND date >= ?''',
        (account_number, five_minutes_ago)
    ).fetchone()

    if recent['cnt'] >= 3:
        conn.execute(
            '''INSERT INTO fraud_alerts
               (user_id, account_number, reason, amount, type,
                from_account, to_account)
               VALUES (?, ?, ?, ?, ?, ?, ?)''',
            (user['id'], account_number,
             'Multiple rapid transactions', amount,
             transaction_type, from_account, to_account)
        )
        conn.execute(
            'UPDATE users SET status = ? WHERE account_number = ?',
            ('suspended', account_number)
        )
        conn.commit()
        return True # Fraud detected - block transaction

    return False # No fraud detected - allow transaction
```


V.2 Atomic Transfer Route (Flask)

```
@app.route('/transfer', methods=['GET', 'POST'])
def transfer():
    if not session.get('username'):
        return redirect(url_for('login'))
    if request.method == 'POST':
        to_account = request.form['to_account']
        amount = int(request.form['amount'])
        from_account = session.get('account_number')

        conn = get_db()
        sender = conn.execute(
            'SELECT * FROM users WHERE account_number = ?',
            (from_account,)
        ).fetchone()
        receiver = conn.execute(
            'SELECT * FROM users WHERE account_number = ?',
            (to_account,)
        ).fetchone()

        if not receiver:
            return render_template('transfer.html',
                                   error='Recipient account not found.')
        if from_account == to_account:
            return render_template('transfer.html',
                                   error='Cannot transfer to your own account.')
        if sender['balance'] < amount:
            return render_template('transfer.html',
                                   error='Insufficient balance.')

        if check_fraud(from_account, amount, 'transfer_out',
                       from_account, to_account):
            return render_template('transfer.html',
                                   error='Transaction flagged. Account suspended.')

        # Atomic update - single commit for both accounts
        conn.execute(
            'UPDATE users SET balance = balance - ? WHERE account_number = ?',
            (amount, from_account)
        )
        conn.execute(
            'UPDATE users SET balance = balance + ? WHERE account_number = ?',
            (amount, to_account)
        )
        conn.execute(
            '''INSERT INTO transactions
               (account_number, type, amount, from_account, to_account)
               VALUES (?, ?, ?, ?, ?)''',
            (from_account, 'transfer_out', amount, from_account, to_account)
        )
        conn.execute(
            '''INSERT INTO transactions
               (account_number, type, amount, from_account, to_account)
               VALUES (?, ?, ?, ?, ?)''',
            (to_account, 'transfer_in', amount, from_account, to_account)
        )
        conn.commit()
        return redirect(url_for('dashboard'))
    return render_template('transfer.html')
```